REP COACH · POST-CALL EVALUATION & COACHING

# Phase 2 — Build Something That Runs

Janos Kristof · July 13, 2026

Prepared under NDA · call data anonymized

## Contents


Rep Coach — Product Requirements (PRD)

1. Why this, and who it's for
2. Scope
3. Functional requirements
4. Non-functional requirements
5. Acceptance criteria (map 1:1 to Verification, and to the 2E gates)
6. Success metrics (how we'd know it worked — "if you can't measure it, it didn't work")
7. Open questions (resolved in discovery / the production session)

Rep Coach — System & Product Design

1. System architecture
   File layout (build/)
2. Post-call process flow
3. Data model
4. Rubric & scoring pipeline
   Prompt shape (documented for reproducibility)
5. UX flows (wireframe-level)
6. Interface contracts (the production seam)
7. Synthetic dataset design (generate_data.py, personas.yaml)
8. Build order (vertical slices for the agentic loop)

Rep Coach — Tool Note

- What it solves
- How it works
- How I built it — an agentic SDLC (the meta-proof)
- How it's measured — "if you can't measure it, it didn't work"
- Path to production (concise — the deep dive is the production-readiness session)
- Templating (the Phase-3 hook, proven now — AC8)
- What I'd build next


# Rep Coach — Product Requirements (PRD)

**Phase 2 · Build** · Janos Kristof · *derived from the Phase 1 diagnosis*

> **One-line product.** Rep Coach scores every sales call against a Sensi-tuned rubric, hands the rep two specific fixes for the next call, and hands the manager a triaged coaching queue — without an hour of manual review. It **installs measurement where there is none today.**

This PRD is deliberately short. It exists so the build has acceptance criteria to hit and so the *way* we build it (the agentic loop in `tool_note.md`) has a target. Design lives in `design.md`.

---

## 1. Why this, and who it's for

The Phase 1 diagnosis found one root cause behind the six stated pains: **no closed loop turns each call into tracked, improving rep behavior.** Rep Coach is the cheapest place to install that loop — it *creates the baseline from call one*.

| Persona | Job-to-be-done | What Rep Coach gives them |
|---|---|---|
| **Rep** | "How did my last call go, and what do I fix before the next one?" | A per-call scorecard with evidence, the top-2 focus areas, and recommended micro-content |
| **Manager** | "Who needs coaching this week — without me reviewing every call?" | A team rollup, an auto-flagged escalation queue, and a triaged coaching queue |
| **Enablement / coach** | "What's *systematically* weak across the team?" | Trend dashboards by rep and by rubric dimension over time |

---

## 2. Scope

**In scope (the running prototype):** - Score a speaker-labeled transcript against a **call-type-aware rubric** ( `rubric.yaml` ). - **Hybrid scoring** — deterministic metrics in code + LLM judgment with **verbatim evidence quotes**. - A **user picker** (dropdown of all users) that drives the app — pick a **rep** → that rep's own tabs (**My Calls · Trends · Activity**); pick the **manager** → team tabs (**Calls · Team · Trends · Measurement · Activity**). A **call list → call detail** workflow with a per-row **Details / Process** button. - **Inline Process** — an unscored call is scored live by the real engine on demand and **auto-routes** the coaching loop (session-only; the committed dataset stays reproducible). - **Mock routing** — processing writes Slack/HubSpot actions to a local log (adapter seam), shown as **per-call activity** on the detail page. - A **reproducible, offline, NDA-safe synthetic dataset** (5 reps, 3 call types, the Sensi agent suite) + the one anonymized real call, with seeded score history for the dashboards.

**Mocked (prototype only, real in production → the production-readiness session):** - The **"call-complete" trigger** — a file-drop / button stands in for a Jiminny webhook. - **Slack + HubSpot** — behind a clean adapter interface; prototype impls write locally.

**Out of scope (explicitly deferred):** - Auth / RBAC, multi-tenant, real integrations, real-time ingestion, production orchestration. - Rebuilding conversation intelligence (capture/transcription = **buy**, per `make_vs_buy.md` ).

**Companion deliverable — `tool_note.md` (the assignment's "short note").** The running app is half the Phase-2 deliverable; the note is the other half and **must** cover, concisely: - **What it solves** — the problem from the diagnosis, in one paragraph. - **How it works** — the hybrid scoring loop + rubric-as-config (points to `design.md` ). - **How I built it** — the **agentic SDLC** (implement → verify → adversarial review → iterate); this is an *explicit, graded* assignment ask ("show us how you built it"), not an afterthought. - **Path to production** — a *concise* summary (trigger, data safety, HubSpot surface, human-in-loop); the deep dive is the dedicated production-readiness session. - **What I'd build next** — the near-term roadmap beyond the prototype.

---

## 3. Functional requirements

**FR1 — Score a call.** Given a speaker-labeled transcript + its metadata (rep, product, call type), produce an **evaluation record**: overall score, per-dimension score + **evidence quotes** + a coaching tip, deterministic metrics, flags, and recommended micro-content.

**FR2 — Rubric as config.** All dimensions, weights, and call-type weightings live in `rubric.yaml` — **no code change** to add a dimension or retune weights. Rubric is **versioned**.

**FR3 — Hybrid, explainable scoring.** Deterministic signals (talk/listen ratio, question counts, next-step booked) are computed in code and shown; LLM-judged dimensions must cite **verbatim quotes** from the transcript. Nothing is an unexplained number.

**FR4 — Rep view: call list → detail.** *My Calls* lists the rep's calls (Date · Account · Product · Type · **Overall**; unscored → "—"). Clicking a call opens its **detail**: the visual scorecard (overall + per-dimension bars, expandable evidence), an **overall summary** (the LLM's, or a generated fallback), **Coach's next steps** (top-2 focus fixes + tips merged with recommended micro-content), and the **per-call integration activity**.

**FR4b — Process an unscored call.** A call with no evaluation opens a **Process** button that runs the whole pipeline live (deterministic + LLM scoring, then routing) — the manual stand-in for production's automated post-call trigger. Processed results persist in session state (the committed dataset stays reproducible); the demo ships with **unscored transcripts** so Process has real work.

**FR5 — Manager view.** Team rollup (rep × avg score + trend), an **escalation queue** (calls with extreme gaps auto-flagged), and a **coaching queue** (who to coach this week). A manager can open **any rep's** call detail — from the all-reps *Calls* list or straight from the Team queues.

**FR6 — Trend dashboard.** Scores over time by rep and by dimension, from seeded history — the five rep archetypes produce a readable story (benchmark, ramp curve, plateau, improvement).

**FR7 — Routing (auto).** Processing a call **auto-routes** the coaching loop — a Slack nudge and a HubSpot scorecard + coaching task — to a local log (adapter seam), proving the loop *routes*, not just scores. (No manual "Send to Slack / Create HubSpot" buttons; routing is a consequence of processing, as it would be in production.)

**FR7b — Per-call activity.** Each call's **detail page** shows the integration actions routed for that call — **endpoint** (Slack / HubSpot / …), **type** (nudge / task / scorecard), **timestamp**, and **content**. This makes the integration points visible on the record itself — the prototype stand-in for the HubSpot activity timeline in production.

**FR8 — Measurement panel.** Show **leading/internal** — *adoption* (rec-adoption, coaching tasks completed / followed up, coverage %), *usage* (weekly-active reps, content engagement, return rate), *behavior change* (recommended fixes recur on later calls, flagged-dimension movement) — and **lagging/external** (call-quality trend, ramp-time proxy, conversion proxy), with the honest baseline note.

**FR9 — Templatable (Phase 3 hook).** Pointing the app at a *second* rubric (e.g. a CS-QA variant) scores a different call type with **no code change**.

---

## 4. Non-functional requirements

- **NFR1 · NDA-safe & offline.** Only synthetic text is sent to the LLM; the one real call is anonymized. Runs without network except the LLM call; pre-computed evaluations are cached so the demo is stable.

- **NFR2 · Reproducible.** Fixed seed + baked-in dates (no wall-clock / RNG at runtime). Regenerating data and re-scoring is one documented command each.
- **NFR3 · Explainable.** Evidence quotes + deterministic metrics are always visible next to scores.
- **NFR4 · Fast enough to demo live.** Cached evaluations render instantly; a single live re-score returns in a few seconds.
- **NFR5 · One-person maintainable.** Small, readable modules; config over code; a `.env.example`; no bespoke infra.
- **NFR6 · Model-configurable.** LLM model is a config value (default **Claude Sonnet 5**), swappable without code changes; structured output is schema-validated.

---

## 5. Acceptance criteria (map 1:1 to Verification, and to the 2E gates)

| # | Acceptance criterion | Verifies |
|---|---|---|
| **AC1** | `pip install -r requirements.txt && streamlit run app.py` → app loads | It runs |
| **AC2** | Select a transcript → scorecard renders with per-dimension scores, evidence quotes, tips | FR1, FR3, FR4 |
| **AC3** | Deterministic metrics (talk/listen ratio, question count) match a hand check on one transcript | FR3 |
| **AC4** | Manager view shows the rollup; an extreme-gap transcript surfaces in the escalation queue | FR5 |
| **AC5** | Trend dashboard renders variance across reps/time from seeded history | FR6 |
| **AC6** | Processing a call **auto-routes** a Slack nudge + a HubSpot scorecard + a coaching task to the local log (no manual "Send" buttons — routing is a consequence of Process, per FR7); the routed actions appear as that call's per-call activity | FR7, FR7b |
| **AC7** | Measurement panel shows leading + lagging metrics populated | FR8 |
| **AC8** | **Templating (done):** `rubric_csqa.yaml` (a CS-QA variant) + a support transcript scored by the **unchanged** engine via `verify_ac8.py` → **93/100**, all 6 support dimensions incl. the reused deterministic/hybrid signals. Sales golden-check still passes (the committed weak-archetype calls R4_disc_001/002 score **10/100** and **5/100**, each firing all three pressure flags; a live re-score lands in the same single-digit band — see the invariant below). | FR2, FR9 |
| **AC9** | **Process (live):** opening an **unscored** call shows a Process button → the real engine runs (deterministic + Claude) → the detail renders the scorecard, and the auto-routed Slack + HubSpot actions appear as that call's **per-call activity** | FR1, FR3, FR4b, FR7, FR7b |
| **AC10** | **Daily-use UX:** a **user picker** selects a rep (→ own **My Calls · Trends · Activity** tabs) or the manager (→ **Calls · Team · Trends · Measurement · Activity** tabs); calls open via a per-row **Details / Process** button; a manager can open any rep's call detail; **no left sidebar** | Usability, FR4, FR5 |

> **Gate note (why AC9/AC10 exist).** These two came from a **human click-through**, not the automated gates — and that is the point. The engineering gate proves the tool *runs correctly*; the content-validation gate is a **text-reading reviewer** that cannot feel a sidebar or notice that scoring is pre-computed. AC1–AC8 were all satisfied by rendering cached evaluations, so the "would a rep actually *use* this daily?" gap slipped through. **AC9/AC10 must be verified by a human or a Playwright click-through**, not the text gate — they close that blind spot.

**Golden-transcript check (build-time):** the anonymized real call must score **low** on exactly the dimensions it fails — *discovery quality, buying-signal handling, proof-point fidelity, next-step specificity* — and the pressure/authenticity flag must fire on the manufactured urgency. If a known-bad call scores well, the rubric is wrong.

> **Templating note (AC8 finding).** Building the CS-QA variant surfaced two remaining sales-isms hardcoded in the LLM prompt — the evaluator **role** and the **flag list**. Both were generalized to read from the rubric (`evaluator_role` + `flags.types`), a **one-time** engine change that makes the loop *fully* rubric-driven: dimensions, weights, motion, proof, role, and flags are all config now. After that, a new team is genuinely a new YAML, not new code — exactly what the templating slice is meant to prove (and to surface). The CS-QA rubric reuses the engine's deterministic/hybrid dimension **keys** (`talk_listen_balance`, `discovery_quality`, `next_step`) because those signals transfer to support QA; only the labels/descriptions change.

---

## 6. Success metrics (how we'd know it worked — "if you can't measure it, it didn't work")

The three internal signals the assignment names — **adoption, usage, behavior change** — are tracked distinctly; the external signals are the outcome metrics.

**Leading — internal (weeks):** - **Adoption** — recommendation-adoption rate; **coaching tasks completed / followed up** (the HubSpot to-dos actually closed); coverage % (share of calls actually evaluated). - **Usage** — **weekly-active reps**; **training / coaching-content engagement** (recommended micro-content opened from a scorecard); **return rate** (do reps come back to their scorecard?) — the direct counter to the falsification "reps open it once and never return." - **Behavior change** — **recommended fixes recur on later calls** (the coaching actually changes behavior); flagged-dimension score movement per rep after coaching.

**Lagging — external (quarter): call quality** (average rubric-score trend per rep), **ramp-time** to full productivity, **conversion / win-rate** by stage.

**Baseline note (honest):** none of these are measured today — Rep Coach **creates the baseline from call one**. The prototype's measurement panel demonstrates the metric definitions on synthetic data; production wires the lagging metrics to HubSpot's real outcome data.

---

## 7. Open questions (resolved in discovery / the production session)

- Are Jiminny's own scorecard/tracker features licensed and sufficient for the rubric? → decides how much of the loop is *build* vs *configure* (`make_vs_buy.md`).

- HubSpot tier (UI Extensions / custom objects / Operations Hub availability) → shapes the production write-back surface.

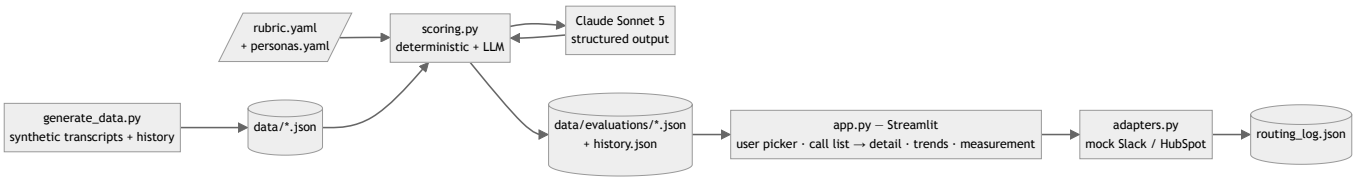
# Rep Coach — System & Product Design

**Phase 2 · Build** · Janos Kristof · *produced before writing app code*

This is the design the agentic build loop implements against. Diagrams here are the **source** for the architecture/process-flow visuals reused in the capstone deck. Scope and acceptance live in `prd.md`; the make-vs-buy stance in `make_vs_buy.md`.

---

## 1. System architecture



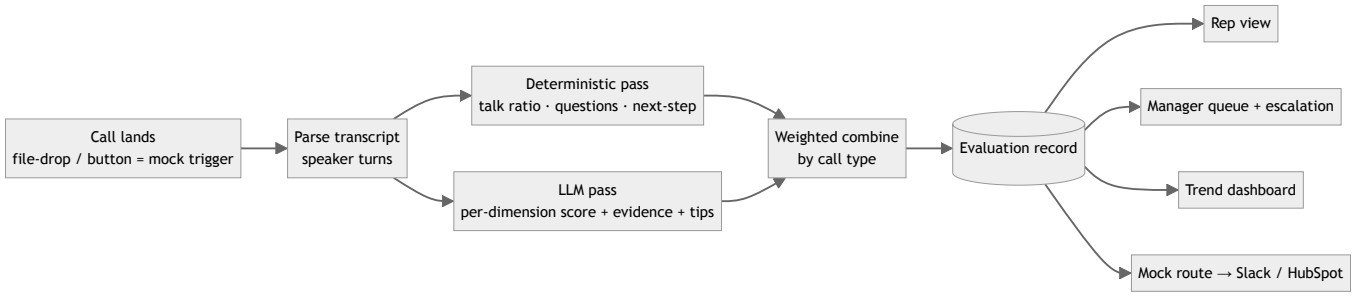

**Design intent:** capture/transcription is *bought* (a transcript comes in); Rep Coach is the **loop on top**. The scoring core is pure Python + a single structured LLM call, so it's testable and CI-tool-agnostic. The UI reads pre-computed evaluations for a stable demo and can re-score one call live.

### File layout ( `build/` )

```
app.py            Streamlit UI (user picker → call list → detail · trends · measurement · activity · routing)
scoring.py        deterministic metrics + LLM scoring + weighted combine (rubric-driven)
rubric.yaml       call-type-aware dimensions & weights (versioned)  ← the templatable seam
rubric_csqa.yaml  Phase-3 CS-QA variant (proves AC8: no code change)
personas.yaml     5 rep archetypes + manager · products · call types
generate_data.py  builds synthetic transcripts + seeded history (fixed seed, baked dates)
adapters.py       Notifier / CRM interfaces + mock implementations + activity seeding
verify_ac8.py     templating proof: same engine scores a support call via rubric_csqa.yaml
verify_ui.py      Playwright end-to-end UI check (fresh server per view, screenshots)
data/
  synthetic/*.json    12–15 speaker-labeled transcripts + 1 anonymized real call
  evaluations/*.json  pre-computed evaluation records (seed the views)
  history.json        ~6–8 weeks of seeded score-summaries for the dashboards
  routing_log.json    mock Slack/HubSpot sink
requirements.txt · .env.example (ANTHROPIC_API_KEY; real .env gitignored)
```

---

## 2. Post-call process flow




---

## 3. Data model

**Transcript** ( `data/synthetic/<call_id>.json` )

```
{
  "call_id": "R3_disc_014",
  "rep_id": "R3", "product": "Sally", "call_type": "discovery",
  "date": "2026-05-18", "duration_min": 42,
  "turns": [
    {"speaker": "Rep", "text": "..."},
    {"speaker": "Prospect", "text": "..."}
  ]
}
```

**Evaluation record** ( `data/evaluations/<call_id>.json` )

```
{
  "call_id": "R3_disc_014", "rep_id": "R3", "product": "Sally",
  "call_type": "discovery", "date": "2026-05-18",
  "rubric_version": "1.0.0", "model": "claude-sonnet-5",
  "overall": 58,
  "deterministic": {"talk_ratio": 0.86, "rep_questions": 3, "open_questions": 1,
                    "next_step_booked": false, "monologue_max_words": 640},
  "dimensions": [
    {"key": "discovery_quality", "score": 2, "weight": 0.22, "signal": "hybrid",
     "evidence": ["\"Any questions from our first conversation?\" → \"I don't have any.\""],
     "tip": "Open with 2–3 fresh discovery questions before any screen-share."}
  ],
  "flags": [{"type": "manufactured_urgency",
             "evidence": ["\"valid until the end of the month\""]}],
  "recommendations": [{"title": "Discovery-first opening (Sally)", "why": "Talk ratio 86%; no fresh discovery"
  "generated_at": "2026-05-18"
}
```

**History record** ( `data/history.json` , seeded — no transcript, cheap depth for trends)

```
{"rep_id": "R5", "date": "2026-04-06", "call_type": "demo_close",
 "product": "Olly", "overall": 71,
 "dimensions": {"discovery_quality": 3, "proof_fidelity": 4, "next_step": 4}}
```

**Rubric** ( `rubric.yaml` ) — see §4.

---

## 4. Rubric & scoring pipeline

The rubric encodes what a good Sensi call looks like, drawn straight from the Phase-1 diagnosis (the real call's failure modes) and Sensi's **own** prescribed motion + published proof wall.

| Dimension | What it measures | Signal | disc / demo / follow weight |
|---|---|---|---|
| **motion_adherence** | Followed Sensi's motion (discovery → product → value) for this call type | LLM | 0.15 / 0.15 / 0.10 |

| Dimension | What it measures | Signal | disc / demo / follow weight |
|---|---|---|---|
| **discovery_quality** | Fresh, open discovery vs. assumptions / leading questions | hybrid | 0.22 / 0.10 / 0.12 |
| **talk_listen_balance** | Rep vs. prospect airtime (monologue penalty) | deterministic | 0.13 / 0.10 / 0.10 |
| **buying_signal** | Detected *and* qualified buying/risk signals | LLM | 0.15 / 0.12 / 0.16 |
| **objection_handling** | Surfaced, acknowledged, resolved objections | LLM | 0.10 / 0.15 / 0.16 |
| **proof_fidelity** | Claims match Sensi's published proof; no invented numbers | LLM | 0.10 / 0.20 / 0.10 |
| **next_step** | Firm, dated next commitment booked | hybrid | 0.15 / 0.18 / 0.26 |
| **authenticity_flags** | Manufactured urgency / false-agreement (negative) | LLM flag | penalty / penalty / penalty |

*Scores are 0–5 per dimension; overall = weighted sum rescaled to 0–100. Weights are per call type and live in `rubric.yaml`; `authenticity_flags` applies a capped penalty rather than a weight.*

**Deterministic pass (in code — not an LLM guess):** - `talk_ratio` = rep_words / total_words → mapped to a 0–5 score (band: ~0.4–0.55 ideal for discovery → 5; > 0.8 monologue → ≤ 2). - `rep_questions` / `open_questions` — count `?` in rep turns; open = starts with what/how/why/tell me/walk me/what if. - `next_step_booked` — regex for a scheduled, dated commitment ("let's meet", a weekday, a date). - `monologue_max_words` — longest single rep turn (feeds the talk-balance narrative).

**LLM pass (one structured call):** Claude receives the rubric definitions, **Sensi's prescribed motion**, **Sensi's published proof wall** (the real numbers, so it can catch invented ones), and the transcript. It returns **only** the LLM/hybrid dimensions with `score`, **verbatim** `evidence` **quotes**, and a concrete `tip`, plus `flags` and `recommendations`. Structured output is enforced via Anthropic **tool-use** (a single `submit_evaluation` tool whose input schema is the record) so the return is always valid JSON — the model retries on schema mismatch.

**Combine:** deterministic dimensions get their computed 0–5 score; hybrid dimensions blend the computed signal with the LLM score (e.g. `discovery_quality` = mean(open-question score, LLM score)); overall = weighted sum by call type, minus flag penalties, rescaled to 0–100.

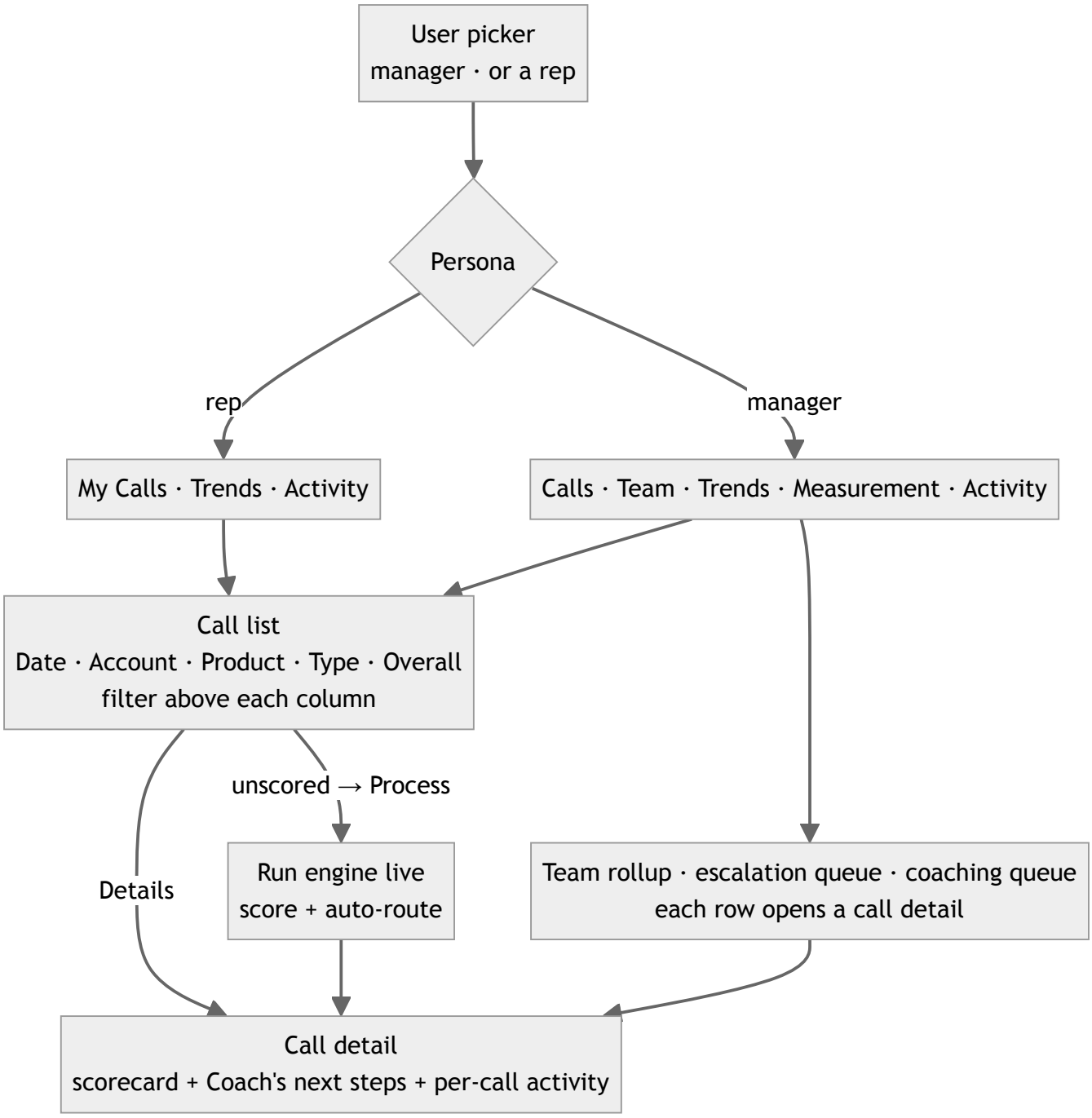
### Prompt shape (documented for reproducibility)

```
SYSTEM: You are a senior {rubric.evaluator_role} evaluator for Sensi (Care Intelligence for senior
        care).  ← role, motion, proof wall, and the flag set are all read from the rubric, so the
        same prompt scaffold serves sales or CS-QA (see AC8). Score only against the rubric. Cite
        VERBATIM quotes from the transcript as evidence for every score. Be calibrated: a monologue
        with no discovery is a 2, not a 4. Flag the rubric's authenticity flags. Recommend Sensi
        micro-content.  Sensi's prescribed motion: <rubric.motion>. Published proof wall: <rubric.proof_wall>.
USER:   <rubric dimensions + call_type weights> + <transcript turns>
TOOL:   submit_evaluation(dimensions[], flags[], recommendations[], summary)   ← schema-enforced
```

---

## 5. UX flows (wireframe-level)

A single **user picker** at the top drives the whole app — pick a **rep** → that rep's own tabs; pick the **manager** → team tabs across everyone. The core interaction is **call list → call detail** (no sidebar).




- **Call list** — Date · Account · Product · Type · **Overall** (unscored → "—"), with a filter control sitting directly above each column; a per-row **Details** button opens the detail (or ⚡ **Process** for an unscored call). The manager's list adds a **Rep** column and spans all reps.
- **Call detail** — the visual **scorecard** (per-dimension score chips + inline evidence + tips), an **overall summary**, **Coach's next steps** (top-2 focus fixes + tips merged with recommended micro-content), the full transcript, and the **per-call integration activity** (the routed Slack/HubSpot actions for *this* call — the prototype view of the HubSpot activity timeline).

- **Process (unscored)** — the ⚡ button runs the real engine end-to-end and **auto-routes** the coaching loop; results persist in session only, so the committed dataset stays reproducible. This is the manual stand-in for production's automated post-call trigger.
- **Team (manager)** — sortable rollup; the escalation queue auto-flags calls where a core dimension is very low or a flag fired; the coaching queue ranks reps. Every entry **drills into the call detail**.
- **Trends** — line charts from `history.json` with a metric selector (Overall + each dimension); the five archetypes make the story legible (R1 benchmark, R3 ramp, R4 plateau, R5 improvement).
- **Measurement** — leading vs. lagging metrics (from `prd.md` §6), split into "computed from evaluations today" vs. "wired to product telemetry in production," with the honest baseline note.
- **Activity** — a filterable feed of every routed action (endpoint · type · rep · product · account · call · timestamp · content) from `routing_log.json`; each synthetic call carries an `account` (a home-care agency) so actions are attributable.

---

## 6. Interface contracts (the production seam)

Routing lives behind narrow interfaces so the production swap is an adapter/config change, not a rewrite — the same seam that becomes **HubSpot as the single surface** in production.

```
class Notifier(Protocol):
    def send_nudge(self, rep_id: str, message: str, link: str) -> None: ...

class CRM(Protocol):
    def create_task(self, owner: str, title: str, due: str, related_to: str) -> str: ...
    def attach_scorecard(self, call_id: str, evaluation: dict) -> None: ...
```

- **Prototype impls** (`MockNotifier`, `MockCRM`) append to `routing_log.json` and surface in the UI — proving the loop *routes*.
- **Production impls** (documented, not built here) — `SlackNotifier`, `HubSpotCRM` (UI Extension card on the call record + Workflows-routed tasks). Selected by config; **the core never changes.**

---

## 7. Synthetic dataset design (`generate_data.py`, `personas.yaml`)

- **5 rep archetypes** → a dashboard story: **R1** top/stable (benchmark) · **R2** solid core · **R3** ramping (low → rising) · **R4** struggling veteran (chronic monologue/leading, plateau) · **R5** improving-after-coaching (clear upward curve).
- **Products** — the Sensi agent suite (Sally – Growth, Olly – Ops, the care co-pilot, + a platform bundle), configured in `personas.yaml`, to exercise **proof_fidelity** scoring.
- **Call types** — discovery / demo-close / follow-up (rubric is call-type-aware).
- **(a)** 14 speaker-labeled full transcripts spanning the quality spectrum (10 live-scored + 4 unscored demo calls), re-using the real call's failure modes; one is the **anonymized real call**, the golden-bad example.
- **(b)** Run the evaluator over that set → real evaluation records seed the views.
- **(c)** ~60–80 seeded history records give the trends depth.

- **Reproducible & offline** — fixed seed, baked-in dates; regeneration + re-scoring each a single documented command.

---

## 8. Build order (vertical slices for the agentic loop)

`rubric.yaml + schemas` → deterministic metrics → LLM scoring (structured) → combine/store → Rep view → Manager view → Trend → Measurement panel → mock routing → **CS-QA rubric variant (AC8)**. Each slice: implement → **verify in the running app** → adversarial **code-review** → fix → next.

# Rep Coach — Tool Note

**Phase 2 · Build** · Janos Kristof · *the companion note to the running app in* `build/`

> **One line.** Rep Coach scores every sales call against a Sensi-tuned rubric, hands the rep two specific fixes for the next call, and hands the manager a triaged coaching queue — **without an hour of manual review.** It installs measurement where there is none today.

This note covers, briefly: **what it solves · how it works · how I built it (the agentic SDLC) · how it's measured · the path to production · what I'd build next.** Requirements are in prd.md; architecture, schemas, and UX flows are in design.md.

---

## What it solves

The Phase-1 diagnosis funnelled Sensi's six stated pains to one root cause: **no closed loop turns each call into tracked, improving rep behavior.** Sensi generates rich signal on every call (a full Jiminny transcript) and throws it away — a measurement vacuum. Managers are the binding constraint: coaching doesn't scale because reviewing calls is manual. Rep Coach is the cheapest place to install the loop — it **creates the baseline from call one** and offloads the review from the manager.

The bet (Phase 1): automate **post-call evaluation + coaching**. One build moves the most pains at once — it directly solves QA, manager load, and rep support, and unlocks knowledge, ramp, and stack consolidation. This tool is that build.

---

## How it works

A post-call loop, hybrid and explainable by design (detail in design.md):

- **Signal → rubric → evaluate → recommend → route.** A speaker-labeled transcript is scored against `rubric.yaml` and an evaluation record is produced.
- **Hybrid scoring — `scoring.py`.** Half the signal is **deterministic, computed in code** (talk/listen ratio, open-question density, next-step-booked, monologue length); the other half is **Claude (Sonnet 5)** via Anthropic **tool-use**, which returns a schema-valid per-dimension score, a **verbatim evidence quote**, and a coaching tip — plus flags and recommendations. *Nothing is an unexplained number:* every LLM score cites the quote it's based on.
- **Rubric-as-config — the templatable seam.** All dimensions, weights (per call type), the prescribed motion, the published proof wall, the evaluator role, and the flag set live in the rubric YAML. **No code change** to retune weights, add a dimension, or stand up a new team (see *Templating* below).
- **Surfaces — `app.py` (Streamlit).** A user picker drives the app: pick a **rep** → their *My Calls · Trends · Activity*; pick the **manager** → *Calls · Team · Trends · Measurement · Activity* across the whole team. A **call list → detail** workflow: each call opens a scorecard (per-dimension chips + evidence + Coach's next steps), the full transcript, and the per-call integration activity. An **unscored** call opens a **Process** button that runs the whole pipeline live — the manual stand-in for production's automated post-call trigger.

- **Mock routing behind an adapter seam — `adapters.py`.** Processing a call **auto-routes** a Slack nudge + a HubSpot scorecard + a coaching task to a local log. The scoring core never talks to Slack/HubSpot directly; it calls narrow `Notifier` / `CRM` protocols, so the production swap is an adapter change, not a rewrite. This is the exact seam that becomes "HubSpot as the single surface" in Phase 3.
- **Reproducible, NDA-safe data — `generate_data.py` + `personas.yaml`.** A fixed seed and baked-in dates (no runtime RNG or wall-clock) produce five rep archetypes that tell a dashboard story — a benchmark, a ramp curve, a plateau, an improvement curve — plus ~7 weeks of seeded history, 10 live-scored transcripts, and 4 unscored demo calls. Only synthetic text is ever sent to the LLM; the one real call is anonymized.

---

## How I built it — an agentic SDLC (the meta-proof)

**I built this the way this role operates: one person + AI agents doing what used to take an ops team.** Phase 2 ran as a compressed, documented software lifecycle, not an ad-hoc build:

> **requirements (prd.md) → design (design.md) → an agentic loop of implement → verify in the *running* app → adversarial review → iterate to the acceptance criteria.**

Built in **vertical slices** (rubric+schema → deterministic metrics → LLM scoring → store → rep view → manager view → trends → measurement → routing → templating), each slice run through the loop until it met its PRD acceptance criterion. What made it real:

- **Verify against behavior, not unit tests alone.** A committed Playwright suite (`verify_ui.py`) drives the *running* app on a fresh server per view and **screenshots every surface** — including a live **Process** run that scores an unscored call end-to-end — so "it works" means the pixels are right, not just that a function returned. The screenshots were read back and drove the next iteration. (This gate re-ran it green: the three cached-path scenarios pass, and the live Process scenario renders the scorecard in ~16s.)
- **A golden-transcript check.** A known-bad call must score **low on exactly the dimensions it fails** and fire the manufactured-urgency flag. If a bad call scores well, the rubric is wrong. (Re-verified after every scoring change. Because the score is a live LLM judgment it drifts a few points run-to-run, so the *invariant* is what's checked — not a fixed number: the weak archetype lands in the single-digit band and fires all three pressure flags. The committed weak calls score **10/100** and **5/100**; a fresh re-score during this gate came back **2/100** — different number, same verdict.)
- **The loop caught real defects — a few worth naming as evidence:**
- **A demo-blocking stability bug.** The app threw an intermittent "Connection error" in a manual test. No Python traceback → a *native* segfault: bleeding-edge Streamlit 1.59 + pandas 3.0 + pyarrow 25 crashing during Arrow serialization. Fixed by pinning to a verified stable combo (`requirements.txt`) — the kind of bug only a *drive-the-running-app* verify pass surfaces.
- **Robustness to structured-output quirks.** Claude occasionally crammed the whole payload into one string field or returned scalar evidence; the LLM pass gained retries + a concatenated-JSON recovery path + normalization so the return is always usable.

- **The gate has blind spots — and a human caught them.** The automated content gate is a *text-reading* reviewer; it can't feel a sidebar or notice that scoring is pre-computed. A manual click-through did, which is why **AC9 (live Process) and AC10 (daily-use UX)** were *added* to the PRD as human/Playwright-verified criteria. The gate now has teeth it didn't before.
- **Templating surfaced a coupling (AC8).** Standing up a second team (CS-QA) exposed two sales-isms hardcoded in the LLM prompt — the evaluator role and the flag list. Both were generalized to read from the rubric, making the loop *fully* config-driven. The exercise proved reuse **and** removed the thing blocking it.

The point isn't the tool alone — it's that the **operating model** (agents implementing, a harness verifying against real behavior, adversarial review, iterate) is repeatable and is itself the AI-enablement capability the role is about.

---

## How it's measured — "if you can't measure it, it didn't work"

Baked into the tool (Measurement tab), split the way the assignment names them:

- **Leading / internal (weeks): adoption** (recommendation-adoption, coaching tasks completed / followed up, coverage %), **usage** (weekly-active reps, training / coaching-content engagement, return rate — the direct counter to "reps open it once and never return"), **behavior change** (recommended fixes recur on later calls, flagged-dimension score movement per rep).
- **Lagging / external (quarter):** call-quality trend, ramp-time, conversion / win-rate — wired to HubSpot outcome data in production.
- **Honest baseline:** none of this is measured today. Rep Coach **creates the baseline from call one**; the internal metrics are computed from real evaluations, the lagging ones are proxies until the HubSpot wiring exists.

---

## Path to production (concise — the deep dive is the production-readiness session)

- **Trigger.** Replace the manual Process button with a **Jiminny call-complete webhook** — no manual upload.
- **Data safety.** A zero-retention, **BAA** LLM endpoint; PII handling; in-region — NDA/HIPAA posture. Prototype sends only synthetic text.
- **Surface = HubSpot (kills the multi-tool jump, pain #1).** The scorecard is attached to the call record; follow-ups become HubSpot **tasks** (prospect next-steps *and* rep coaching to-dos); rollup/queues/trends are **native HubSpot dashboards**; Slack is a nudge layer that deep-links back. The prototype's adapter seam is exactly where this swaps in.
- **Human-in-loop.** Escalations, rubric changes, and high-stakes coaching stay human.
- **Runs without me.** Versioned rubric, monitoring, one-command deploy, config over code.

Full make-vs-buy, the orchestration decision (HubSpot-native vs. n8n — challenged, not assumed), and the non-functional detail live in `production_readiness/`.

---

## Templating (the Phase-3 hook, proven now — AC8)

The engine is generic; **only the rubric + the signal source change per team.** Proven with a second instance: `rubric_csqa.yaml` (Customer-Support QA on support calls), scored by the **unchanged** engine via `verify_ac8.py` — a support transcript scored **93/100** across six support-specific dimensions, reusing the deterministic/hybrid signals as-is. *For a team whose signals map onto the existing ones, a new team is a new YAML, not new code* — CS-QA reuses the three deterministic keys (agent airtime, probing-question density, a firm follow-up) because those transfer directly to support. The honest boundary: a genuinely *new* deterministic signal (say, hold-time) is a one-time addition to the scoring engine, then it too becomes config — the LLM-judged dimensions, weights, motion, proof wall, evaluator role, and flags are already fully rubric-driven. The Phase-1 "one build → many pains" and this "one engine → many teams" are the same idea on two axes — the compounding argument.

---

## What I'd build next

1. **Wire the real trigger + HubSpot write-back** (Jiminny webhook → scorecard on the call record).
2. **Rubric v2 from outcomes** — once calls link to won/lost and ramp data, tune weights to what actually predicts results, and add a lightweight review step where managers correct scores (the labels that improve the rubric over time).
3. **Close the content loop** — recommended micro-content that adapts to each rep's weakest dimensions, sourced from the product KB.
4. **Second team live** — stand up CS-QA (or onboarding knowledge checks) on real calls to prove the pattern compounds, then make the production-orchestration call.